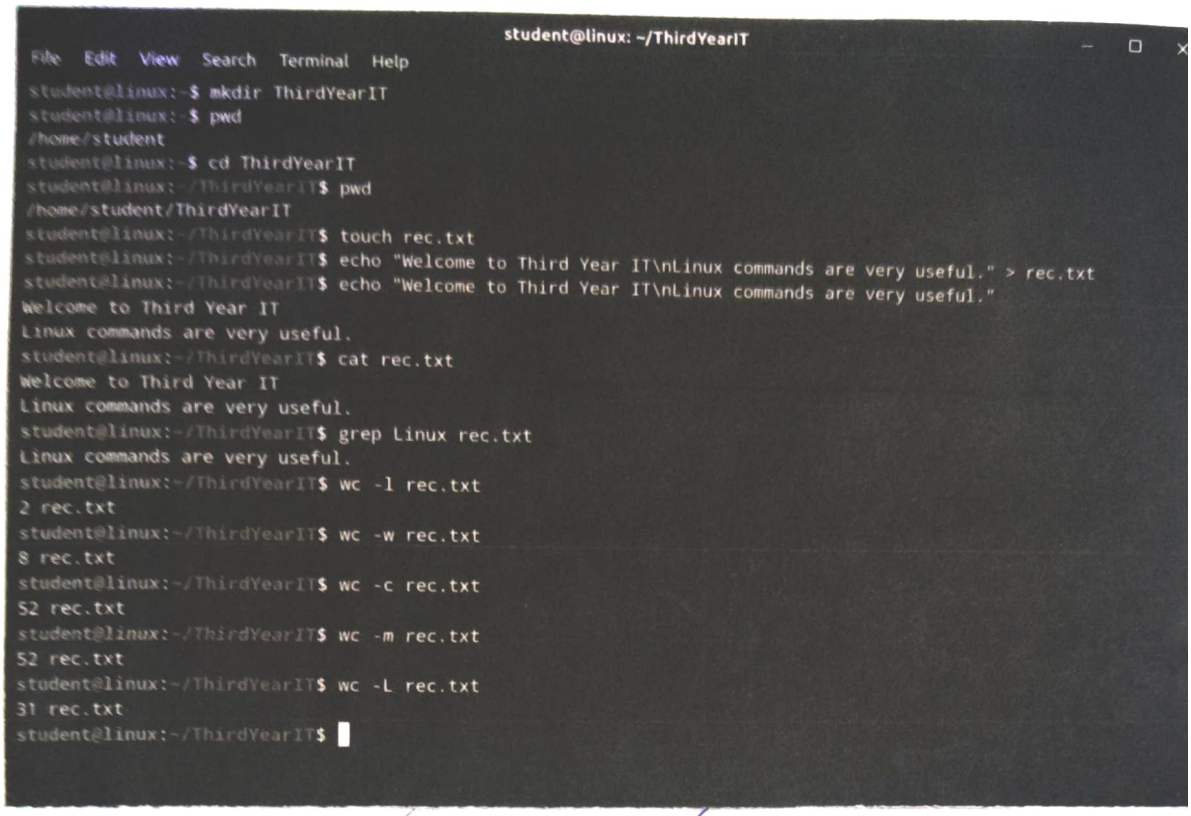
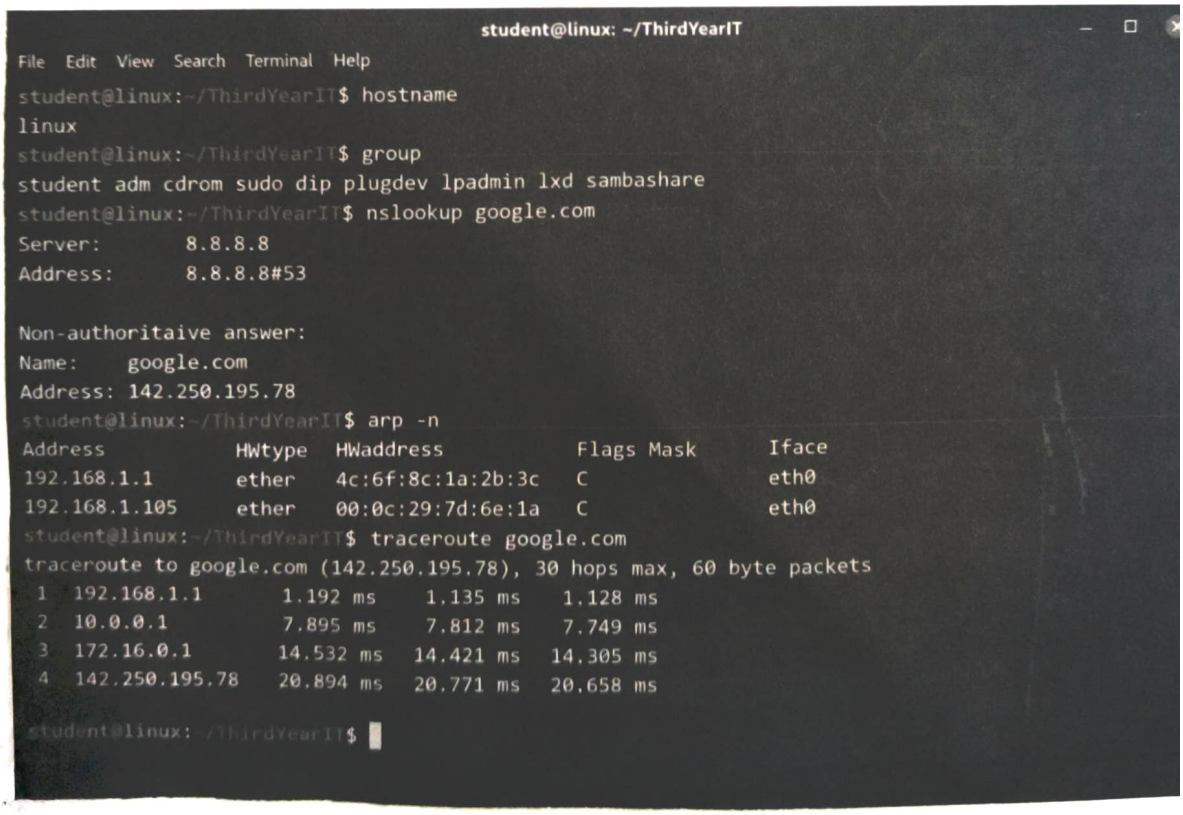
EXP NO: 01

BASIC NETWORKING COMMANDS IN LINUX AND WINDOWS OPERATING SYSTEMS

```
user$ mkdir thirdyearIT
user$ pwd
user$ /tmp/user/home                    -> O/P
user$ cd thirdyearIT
user$ pwd
user$ /tmp/user/home/thirdyearIT        O/P
user$ touch rec.txt
user$ echo "hii this is Sunidhi"
      hii this is Sunidhi               -> O/P
user$ echo "hii" > rec.txt
user$ cat rec.txt
      hii
user$ hostname
      localhost
user$ grep "hii" rec.txt
      hii
user$ wc -l < rec.txt                   -> O/P : 1
user$ wc -c < rec.txt                   -> O/P : 2
user$ wc -w < rec.txt                   -> O/P : 1
user$ wc -m < rec.txt                   -> O/P : 2
user$ wc -L < rec.txt
user$ route -n
user$ nslookup google.com
```


student@linux: ~/ThirdYearIT
File Edit View Search Terminal Help
student@linux:~$ mkdir ThirdYearIT
student@linux:~$ pwd
/home/student
student@linux:~$ cd ThirdYearIT
student@linux:~/ThirdYearIT$ pwd
/home/student/ThirdYearIT
student@linux:~/ThirdYearIT$ touch rec.txt
student@linux:~/ThirdYearIT$ echo "Welcome to Third Year IT\nLinux commands are very useful." > rec.txt
student@linux:~/ThirdYearIT$ echo "Welcome to Third Year IT\nLinux commands are very useful."
Welcome to Third Year IT
Linux commands are very useful.
student@linux:~/ThirdYearIT$ cat rec.txt
Welcome to Third Year IT
Linux commands are very useful.
student@linux:~/ThirdYearIT$ grep Linux rec.txt
Linux commands are very useful.
student@linux:~/ThirdYearIT$ wc -l rec.txt
2 rec.txt
student@linux:~/ThirdYearIT$ wc -w rec.txt
8 rec.txt
student@linux:~/ThirdYearIT$ wc -c rec.txt
52 rec.txt
student@linux:~/ThirdYearIT$ wc -m rec.txt
52 rec.txt
student@linux:~/ThirdYearIT$ wc -L rec.txt
31 rec.txt
student@linux:~/ThirdYearIT$


student@linux: ~/ThirdYearIT
File Edit View Search Terminal Help
student@linux:~/ThirdYearIT$ hostname
linux
student@linux:~/ThirdYearIT$ group
student adm cdrom sudo dip plugdev lpadmin lxd sambashare
student@linux:~/ThirdYearIT$ nslookup google.com
Server: 8.8.8.8
Address: 8.8.8.8#53
Non-authoritaive answer:
Name: google.com
Address: 142.250.195.78
student@linux:~/ThirdYearIT$ arp -n
Address HWtype HWaddress Flags Mask Iface
192.168.1.1 ether 4c:6f:8c:1a:2b:3c C eth0
192.168.1.105 ether 00:0c:29:7d:6e:1a C eth0
student@linux:~/ThirdYearIT$ traceroute google.com
traceroute to google.com (142.250.195.78), 30 hops max, 60 byte packets
1 192.168.1.1 1.192 ms 1.135 ms 1.128 ms
2 10.0.0.1 7.895 ms 7.812 ms 7.749 ms
3 172.16.0.1 14.532 ms 14.421 ms 14.305 ms
4 142.250.195.78 20.894 ms 20.771 ms 20.658 ms
student@linux: /ThirdYearIT$

EXP NO : 02
Date : 16/01/26

Study of different types of Network cables and Crimping of cable with RJ45 connector

AIM:

To study the different types of network cables.

a) Understanding different types of network cables.

The different types of cables used in networking are:

1. Unshielded Twisted Pair (UTP) cable
2. Shielded Twisted Pair (STP) cable
3. Co-axial cable
4. Fibre optic cable

Cable Type	Category	Maximum data trans	Adv/ Dis	Applications
UTP	Category 3 Category 5 Category 5e	10 bps upto 100Mbps 1 Gbps	Advantages: → cheaper → easy to install Disadv → More prone to EMI & noise	10 Base - T Ethernet Fast Ethernet
STP	Category 6, 6a	10 Gbps	Adv → shielded → Faster then UTP → Less to noise	Gigabit - Ethernet

Cable type	Category	Maximum data transmission	Adv / disadv	Applications / use
Coaxial cable	RG-6 RG-59 RG-11	10 – 100 Mbps	Adv: → high band width → Versatile → DisAdv → Limited distance → cost → size is bulky	Speed of signal is 500m, Television network
Fibre optic cable	Single mode, multimode	100 Gbps	Adv: → high speed → high bandwidth → high security → Disadv: → Expensive → Requires skilled Installers	Maximum distance of fibre optics cable is 100 meter

RESULT:

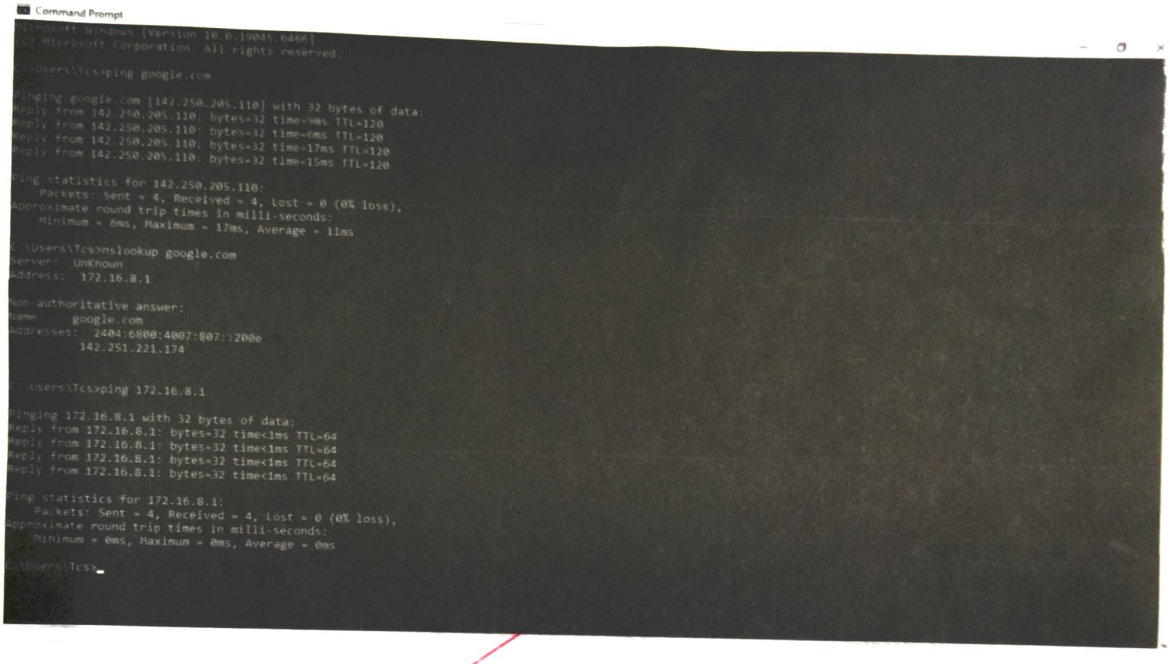
Thus, the different types of network cables were studied & verified

EXP NO: 03
DATE: 18/7/26

Develop a Customized Ping Command To test the server Connectivity

Aim:-

To develop a customized Ping command and to test the server connectivity



Result:-

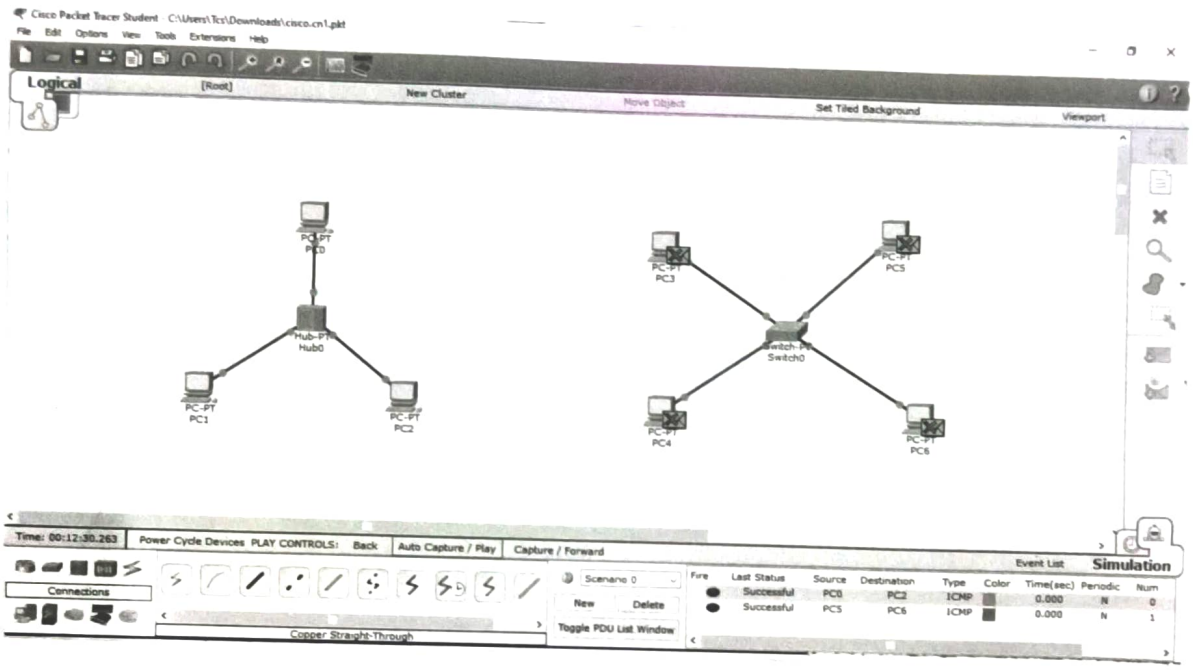
Thus, successfully completed developing a customized ping command and tested the server connetivity.

EXP NO: 4
Date: 22/7/26

Packet Tracer Installation and user interface overview.

Aim:-

To do Packet tracer installation and to study the user interface overview.



Result:-

Thus the installation of packet tracer and the overview of user interface was successfully accomplished.

EXP NO: 05 IMPLEMENTING ECHO using Python

DATE : 23/7/26

AIM:

To implement echo using Python.

```
import socket

HOST = '127.0.0.1'  # Localhost
PORT = 12345

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)

print("Server is waiting for connection...")

conn, addr = server.accept()
print("Connected by:", addr)

while True:
    data = conn.recv(1024).decode()

    if not data or data.lower() == "exit":
        print("Connection closed.")
        break

    print("Client:", data)
    conn.send(data.encode())  # Echo back the same message

conn.close()
server.close()
```



```
import socket

HOST = '127.0.0.1'
PORT = 12345

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))

while True:
    message = input("Enter message: ")

    client.send(message.encode())

    if message.lower() == "exit":
        break

    data = client.recv(1024).decode()
    print("Server:", data)

client.close()
```



```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:\Users\REC\Downloads\client.py ==========
Enter message: HAI
Server: HAI
Enter message: HAI
Server: HAI
Enter message:
```



```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/REC/Downloads/server.py ==========
Server is waiting for connection...
Connected by: ('127.0.0.1', 57676)
Client: HAI
Client: HAI
```



Result:

Thus, implementing echo using python is executed successfully.

Exp No: 06

Date: 30/7/26

BYTE STUFFING USING PYTHON

AIM:

To implement byte stuffing using Python.

byte_stuffing.py - C:/Users/REC/AppData/Local/Programs/Python/Python310/byte_stuffing.p...

File Edit Format Run Options Window Help

```
# Byte Stuffing with Binary Representation

flag = input("Enter Flag Character: ")
escape = input("Enter Escape Character: ")
data = input("Enter Data: ")

stuffed = ""

for ch in data:
    if ch == flag or ch == escape:
        stuffed += escape
    stuffed += ch

frame = flag + stuffed + flag

print("\nOriginal Data      :", data)
print("Stuffed Data       :", stuffed)
print("Transmitted Frame  :", frame)

# Function to print binary values
def show_binary(text):
    for ch in text:
        print(ch, "->", format(ord(ch), '08b'))

print("\nBinary of Flag:")
show_binary(flag)

print("\nBinary of Escape:")
show_binary(escape)

print("\nBinary of Original Data:")
show_binary(data)

print("\nBinary of Stuffed Data:")
show_binary(stuffed)

print("\nBinary of Transmitted Frame:")
show_binary(frame)
```

Ln: 39 Col: 0

IDLE Shell 3.10.6

File Edit Shell Debug Options Window Help

```
>>> 
= RESTART: C:/Users/REC/AppData/Local/Programs/Python/Python310/byte_stuffing.py
Enter Flag Character: F
Enter Escape Character: E
Enter Data: AEFB

Original Data       : AEFB
Stuffed Data        : AEEEFB
Transmitted Frame   : FAEEEFBF

Binary of Flag:
F -> 01000110

Binary of Escape:
E -> 01000101

Binary of Original Data:
A -> 01000001
E -> 01000101
F -> 01000110
B -> 01000010

Binary of Stuffed Data:
A -> 01000001
E -> 01000101
E -> 01000101
E -> 01000101
F -> 01000110
B -> 01000010

Binary of Transmitted Frame:
F -> 01000110
A -> 01000001
E -> 01000101
E -> 01000101
E -> 01000101
F -> 01000110
B -> 01000010
F -> 01000110
>>> 
```

Ln: 42 Col: 0

RESULT:

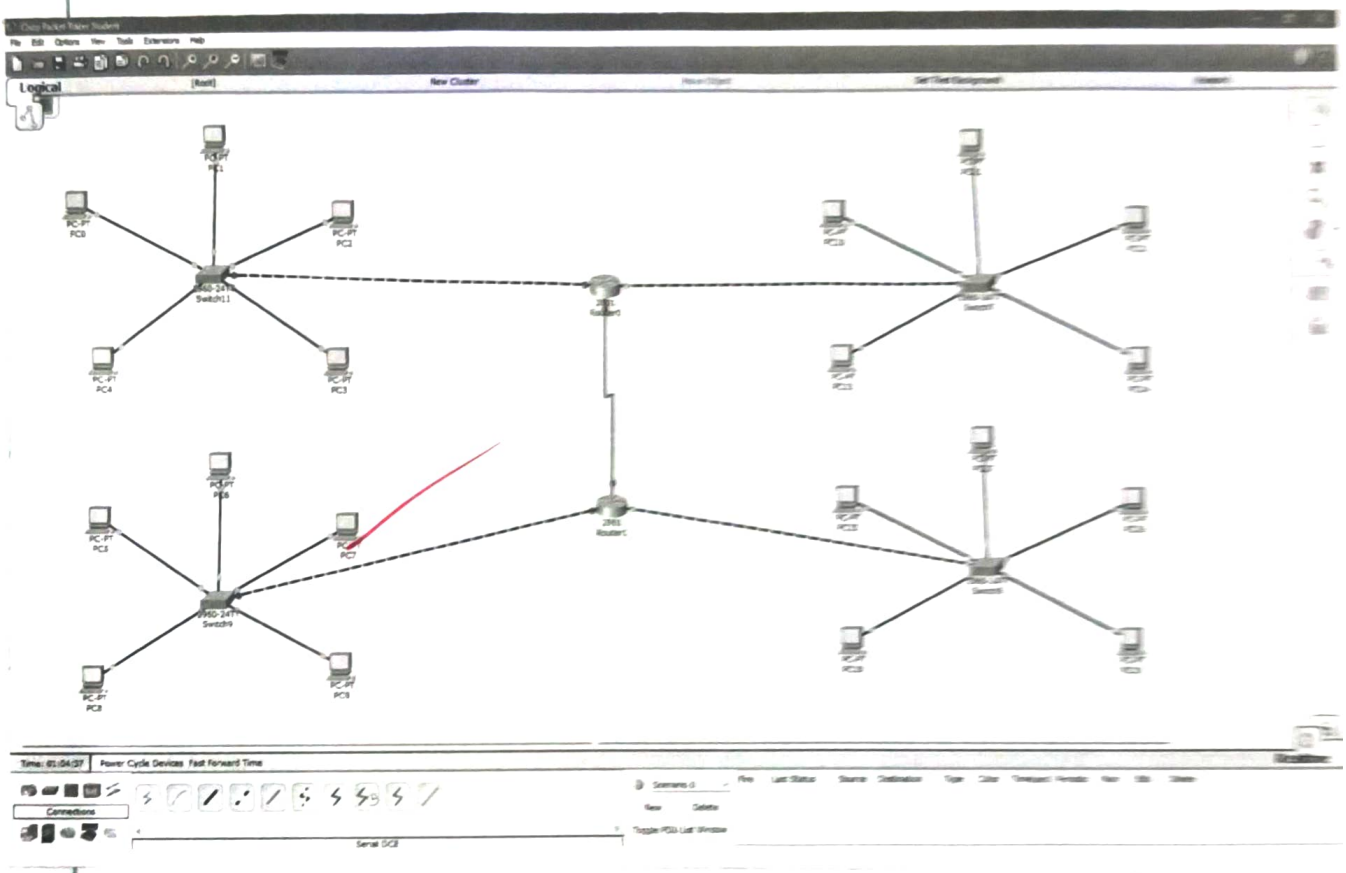
Thus, Byte stuffing using python is executed successfully.

EXP: 07 IMPLEMENTATION OF SUBNETTING IN

Date: 19/8/26 CISCO PACKET TRACER SIMULATOR

AIM:

To implement subnetting in CISCO packet tracker simulator



RESULT:

Thus, Cisco packet tracer simulator is implemented successfully.

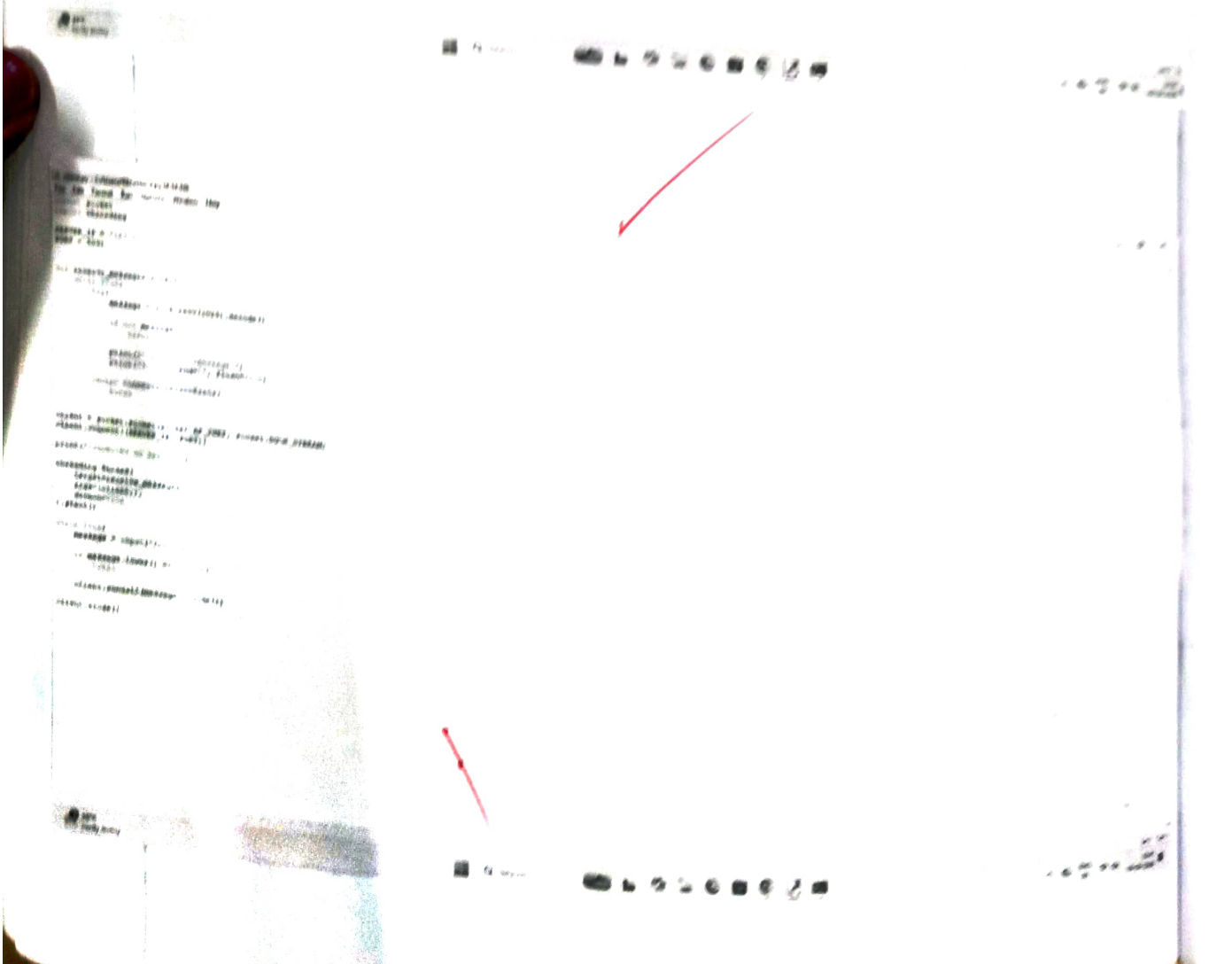
EXP NO : 8 Client Server Communication

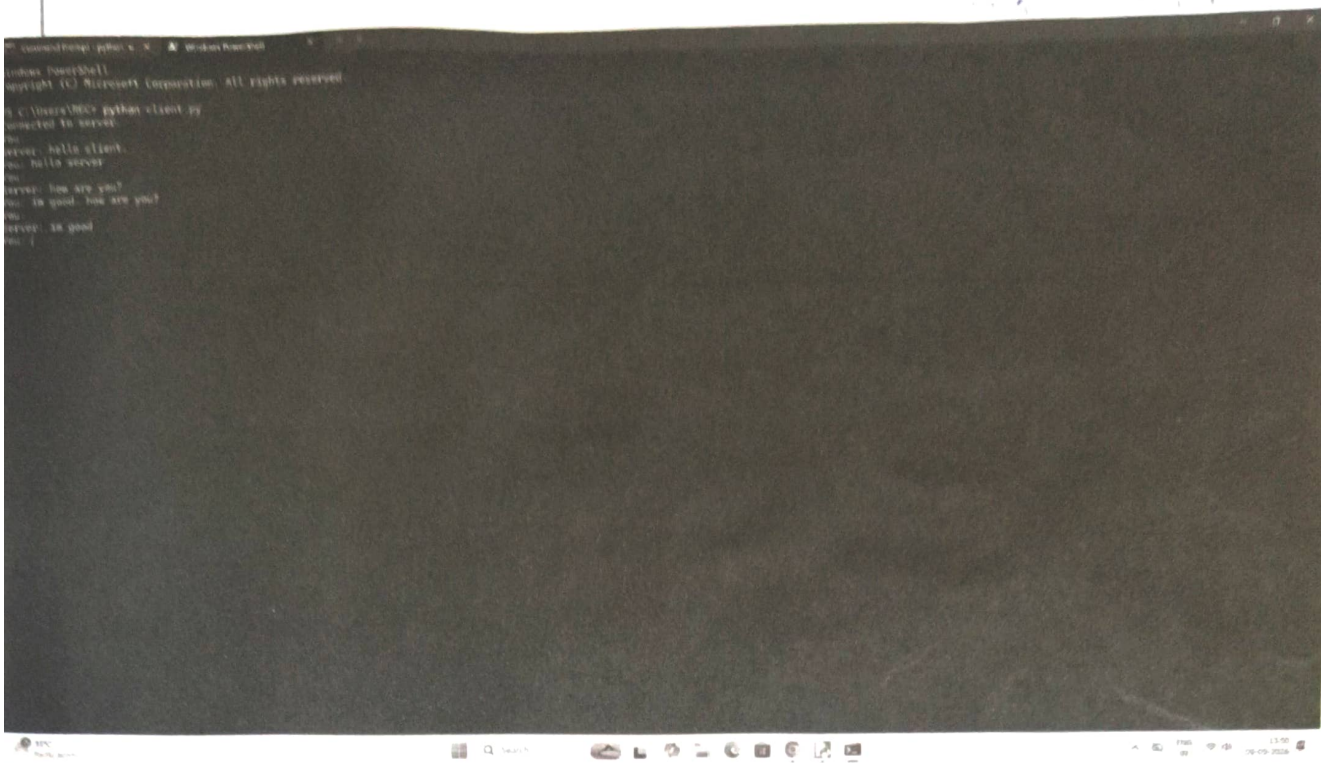
DATE : 9/9/24 using python socket Programming

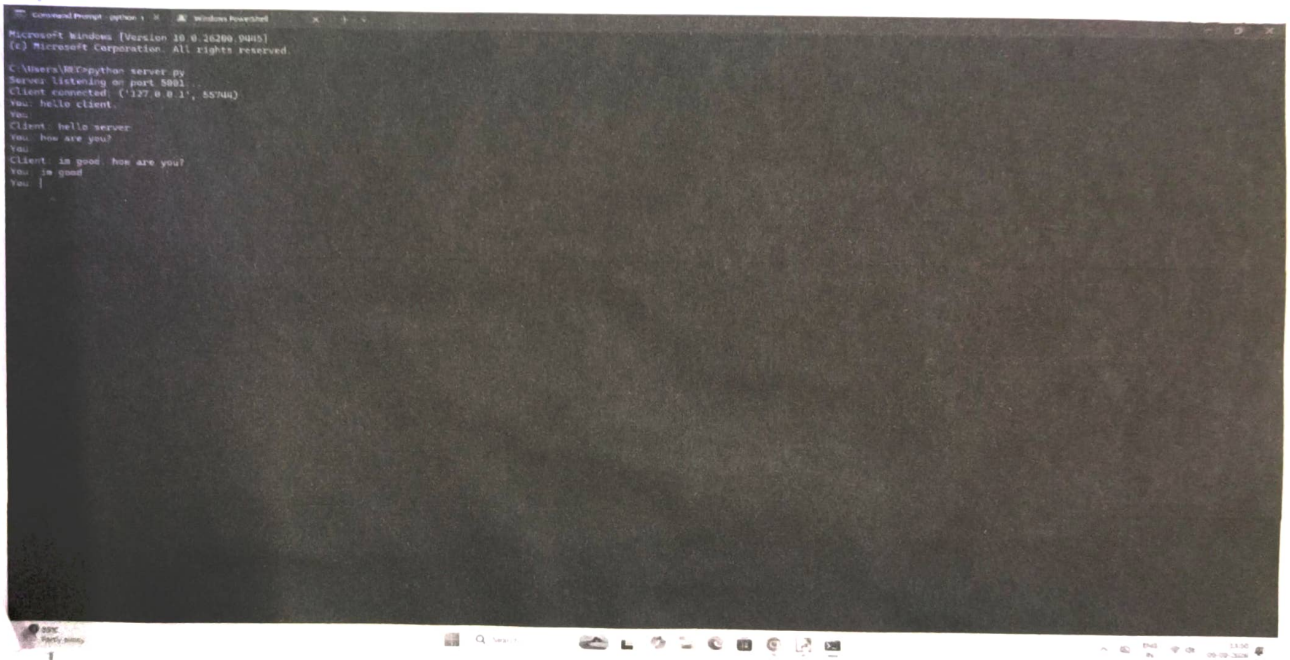
AIM :

To establish communication between client and server using python Programming









RESULT:

Thus the client - server communication was successfully established and message was exchange between client & server.

EXP: 9 DISTANCE VECTOR ROUTING

Date: 23/8/26

AIM: To simulate distance vector routing protocol.

Command Prompt - python

```
...     net.display_tables()

Simulating Distance Vector Routing Protocol...
Network converged in 3 iterations.

--- Routing Table for Router A ---
Destination     Cost        Next Hop
A               0           A
B               2           B
C               3           B
D               5           B

--- Routing Table for Router B ---
Destination     Cost        Next Hop
A               2           A
B               0           B
C               1           C
D               3           C

--- Routing Table for Router C ---
Destination     Cost        Next Hop
A               7           A
B               1           B
C               0           C
D               2           D

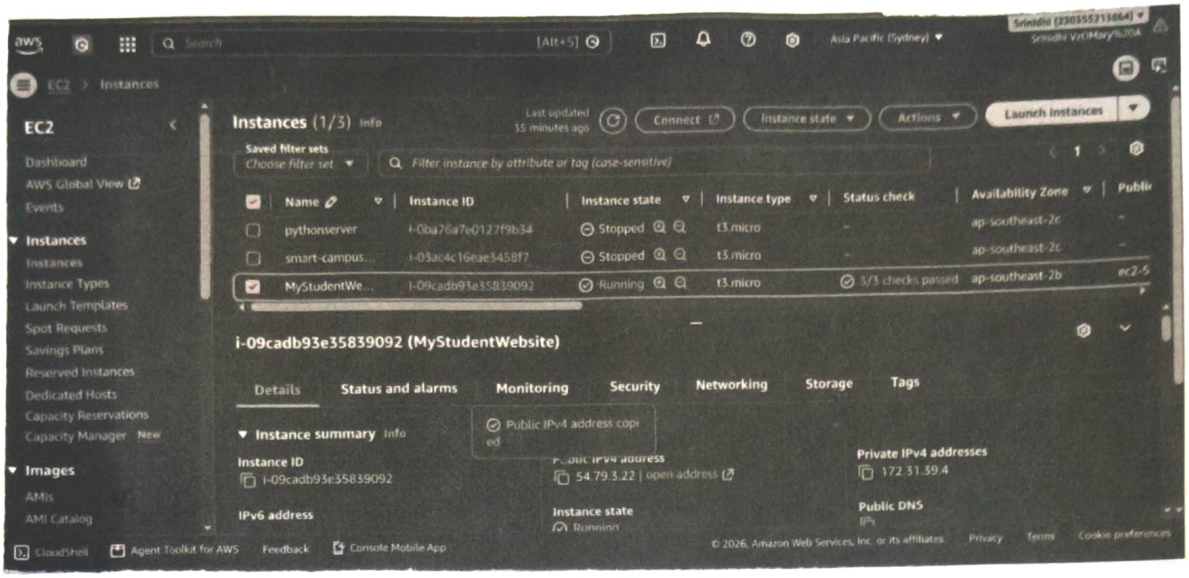
--- Routing Table for Router D ---
Destination     Cost        Next Hop
A               inf         -
B               4           B
```

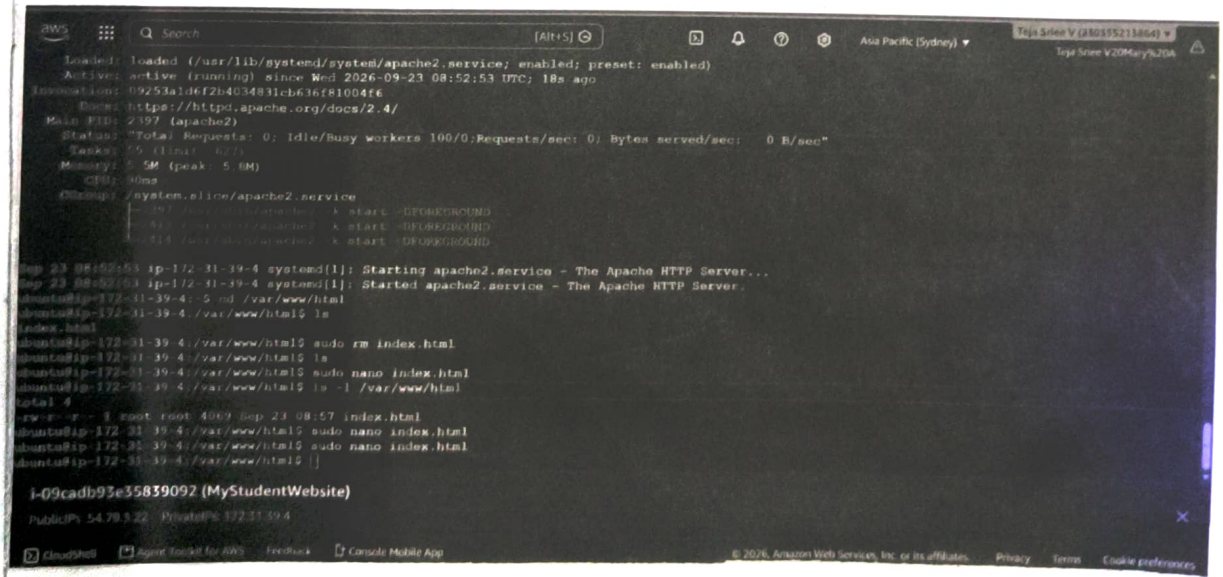
RESULT:

Thus, the distance vector routing protocol was simulated.

EXP NO: 10
DATE: 23/9/26

DEPLOYING WEB APPLICATION ON AN AWS CLOUD SERVER.







Result:

Thus, the web application is successfully deployed on an AWS cloud server.

Exp: 11
Date: 20/8/26

EXPERIMENT ON PACKET CAPTURE TOOL - WIRESHARK

AIM:

Capturing on analysis packets using wireshark tool.

(1) ⇒ Create a filter to display only TCP/UDP packets, inspects the packets and provide the flow graph

⇒ PROCEDURE:

⇒ Select LAN connections in wireshark

⇒ Go to capture → Option

⇒ Select stop capture automatically after 100 packets

⇒ Save the packets

(2) AIM::

Create a filter to display only ARP packets and inspects the packets.

PROCEDURE:

⇒ Go to capture

⇒ Set stop capture automatically after 100 packets

⇒ Then click start capture

⇒ Save the packets.

(3) AIM:

Create a filter to display only DNS packets and provide the flow graph.

PROCEDURE:

* Go to capture → option
* Select stop capture automatically after 100 packets
* Then click start capture
* Search DNS packets
* Save the packets.

4. AIM:

Create a filter to display only HTTP packet & repeate the packets

PROCEDURE:

→ Select LAN connection in Wireshark
→ Go to capture → option
→ Then click start capture
→ Select HTTP packets in search
→ Save the packets.

5. AIM:

To create a filter to display only IP/ICMP packet and inspect the packet

PROCEDURE:

⇒ Select a LAN connection in Wireshark
⇒ Go to capture → option
⇒ Click start capture
⇒ Search for ICMP/IP packets
⇒ Save the packets.

6) AIM:

Create a filter to display only SMCP packet & inspect the packets

PROCEDURE:

⇒ Select LAN connection in wireshark.

⇒ Go to capture → option

⇒ Select the stop capture automatically

⇒ Save the packets

RESULT:

Thus, network packets were successfully captured and analysed using wireshark with different protocol filters

Exp: 12	DISCOVERY OF LIVE HOST USING
Date: 17/9/26	ARP, ICMP, TCP, UDP

AIM:

To discover live host in a network using Nmap scans such as ARP, ICMP, TCP and UDP on the Try Hackme platform.

OBJECTIVE:

To understand how nmap identifies active system before performing further network scanning.

SOFTWARE:

Nmap, Tryhack Nmap loom,
Computer with network success

THEORY:

Nmap is a free and open source network scanning tools used to discover live hosts and identify network services
It was created
Nmap uses different live hosts using different protocols.

1) ARP Scan - Uses ARP scans request to find hosts on same subnet

2) ICMP Scan - Use ICMP request and replies to identify active host

(3) TCP scan - Use TCP requests and packets to specified ports & check the response.

(4) UDP scan - Send UDP packets & can identify an active host through an ICMP port

OBSERVATION:

→ ARP request can discover hosts within the same subset

→ ARP cannot cross routers because it operates at the link layer.

→ An ARP ping generally requires a previledged account

→ UDP ping can identify an active host when an ICMP port unreachable is recived

→ The -Sn option allows Nmap to perform host discovery without conducting a port scan

RESULT:

Thus, the live host discovery using Nmap was successfully studied and performed. The experiment helped in understanding how active hosts can be identified before performing further network scans.

Exp no: 13
Date: 27/8/26

SLIDING WINDOW PROTOCOL

AIM:

To write a program to implement flow control at the data link layer using the sliding window Protocol and simulate the flow of frames from one node to another.

SENDER CODE:

```
message = input("Enter the message : ")
window_size = int(input("Enter window size : "))
frames = []
for i, ch in enumerate(message):
        frames.append((i+1, ch))
print("\n Frames created")
for no, data in frames:
        print(f"Frames {no} : {data}")
with open("Sender_Buffer.txt", "w") as file:
    for no, data in frames:
        file.write(f"{no} {data}\n")
    print("\n Frames stored in Sender_Buffer.txt")
base = 0
While base < len(frames):
        print("\n Current window")
```

```
        for i in range(base, min(base + window_size,
        len(frames))):
            print(f"[{frames[i][0]}: {frames[i][1]}]",
                  end=" ")

    try:
        with open("Receiver_Buffer.txt", "r") as file:
            response = file.readline().strip()
    except FileNotFoundError:
            response = ""

    if response.startswith("ACK"):
            ack = int(response.split()[1])
            print(f"ACK {ack} received")
            base = ack
            open("Receiver_Buffer.txt", "w").close()
    elif response.startswidth("NACK"):
            nack = int(response.split()[1])
            print(f"NACK {nack} received.")
            print(f"Retransmitting frames {nack}...")
            open("Receiver_Buffer.txt", "w").close()
    else:
        print("In Frames sent.")
        input("Run Receiver program & press ENTER to
              continue")
    print("All frames transmitted")
```

RECEIVER CODE:

```
expected_frame = 1
While True:
    try:
        with open("sender_Buffet.txt", "r") as file:
            frames = file.readlines()
    except FileNotFoundError:
            print("sender_Buffet.txt not found")
            break
if not frames:
    print("No frames Available")
    break.
print("In Frames received from sender")
for line in Frames:
        ports = line.split()
        if len(ports) != 2:
                continue
        frame_no = int(ports[0])
        data = ports[1]
        print(f"Frame {frame_no}: {data}")
        if frame_no == expected_frame:
            print(f"Frame {frame_no} received")
```

```
        with open ("Receiver_Buffer.txt", "w") as ack_file:
            ack_file.write (f"ACK {frame_no}")
        print (f"ACK {frame_no} sent")
        expected_frame += 1
    else:
        print ("Wrong frame received")
        with open ("Receiver_Buffer.txt", "w") as ack_file:
            ack_file.write(f"NACK {expected_frame} sent")
        break
if expected_frame > len(frames):
        print ("All frames received")
        break
input ("Press ENTER to check next frame")
```

RESULT:

The sliding window protocol was successfully implemented using python and the data transmission and acknowledgement process was verified successfully.

EXP: 14
Date: 8/10/26

IMPLEMENTATION PACKET SNIFFING USING RAW SOCKETS

AIM:

To implement packet sniffing using RAW sockets in python.

ALGORITHM:

1. Create a RAW socket
2. Bind the socket to the network interface
3. Receiver packets continuously
4. Read the Packets Leader.
5. Display the source & destination IP address
6. display the protocol used
7. Continue until the user stops the programs.

INPUT:

Network packets received through the network interface

CODE:

```
import socket.
S = socket . Socket (socket . IF_INET , socket . Sock_RAW,
                     socket . INPROTO_IP)

S. bind (("0. 0. 0. 0", 0))
S. setSockopt (socket . IPPROTO_IP, socket . IP_HDRINCL
print ("Packet sniffer started . . ")
```

```
While True:
    packet, addr = s.recvfrom(65565)
    ip_header = packet[0:20]
    iph = struct.unpack("!BBHHHBBH4s4s", ipheader)
src_ip = socket.inet_ntoa(iph[8])
dest_ip = socket.inet_ntoa(iph[9])
print("Source IP:", src_ip)
print("Destination IP:", dest_ip)
print("Packet length:", len(Packet))
```

OUTPUT:

```
Packet sniffer started..
Source IP: 192.168.1.10
Destination IP: 192.168.1.1
Packet length: 79
```

RESULT:

Thus, a RAW socket program was successfully implemented to captured & display network packets

EXP NO: 15
Date: 8/10/26

ANALYSE VARIOUS TYPES OF SERVER LOGS USING WEBALIZER TOOL

AIM:

To analyse different types of server logs using the webalizer tool.

ALGORITHM:

1. Install and open webalizer
2. Select the web server log file
3. Configure webalizer to read the log file.
4. Run webalizer to read the log file
5. Open the generated HTML report
6. Analyze visitors, requests, pages, data transfer and HTTP status code

INPUT:

The web server log file such as access log

OUTPUT:

Webalizer generates reports showing

* Number of visitors
* Most visited page
* data transffered
* HTTP status codes.

RESULT:

The web logs were successfully analysed using webanlizer and web traffic statistics was generated.